
TRACE

An On-Device Visual Memory Application for Personal Objects

Technical Report

Final Project - Android Application

Project	TRACE
Application version	0.1.0 (versionCode 1)
Platform	Android 7.0 and later (minSdk 24, targetSdk 36)
Repository branch	final
Report date	5 September 2026

Team identification from repository history

No.	Repository identity	Student ID and official name
1	Khuong_Duong	24125062 - Dương Gia Khương
2	phongcp	24125073 - Nguyễn Trọng Hùng Phong
3	Nguyen Hoang Thuan Phat / Bix2llets	24125072 - Nguyễn Hoàng Thuận Phát
4	ThanDongVanHoc	24125019 - Nguyễn Chí Tính

Overview

This report features TRACE — our final project for the course Mobile Device Application Development — an Android application that records where and when a previously enrolled personal object was last observed. The report documents the problem definition, target users, implemented architecture, algorithms, storage and security design, development environment, installation procedure, verification evidence, work allocation, and completion assessment required by the course specification.

All implementation claims refer to the **final** branch on Github repository, whose links is provided within this project. The final application is a standalone on-device system featuring camera capture, region-of-interest processing, computer vision inference, local authentication, persistence, encryption, location acquisition, and notifications using native features of Android. The **services/api** and **playground** directories are retained as development history and integration harnesses, which serve as our initial point of development and were later faithfully ported as source code of the final product.

Contents

Document Scope and Evidence Base	i
1 Project Topic and Product Problem	1
1.1 Problem statement	1
1.2 Measurable project objectives	1
2 Target Users and Requirements	2
2.1 Primary user groups	2
2.2 Primary usage scenario	2
2.3 Functional and quality requirements	3
3 Application Architecture	4
3.1 Architectural style	4
3.2 Module responsibilities	4
3.3 Control flow and failure propagation	4
3.4 Integrated backend service	5
4 Enrollment and Recognition Pipeline	5
4.1 Reference enrollment	5
4.2 Dual-path scene recognition	5
4.3 Algorithmic strengths and limits	6
5 Memory Logic and Persistent Data	7
5.1 Room schema	7
5.2 Sighting creation and deduplication	7
5.3 Consistency boundary	8
6 Security and Privacy Design	8
6.1 Threat model and trust boundary	8
6.2 Confidentiality under complete storage disclosure	8
6.3 Tamper detection and fail-closed release	9
6.4 Context binding against record substitution	9
6.5 Storage adapters and operational safeguards	10
6.6 Local credential handling	10
6.7 Threat-control matrix	11
7 User Interface and Device Integration	11
7.1 Screen structure and state flow	11
7.2 Runtime permissions	11
7.3 ROI interaction and duplicate-submission control	12
7.4 Accessibility status	12
8 Technology Stack and Engineering Choices	12
8.1 On-device versus client-server processing	13
8.2 Packaging and release strategy	13
9 Setup, Build, and Installation	13
9.1 Required environment	13
9.2 SDK discovery	13
9.3 Canonical commands from repository root	13
9.4 Install and smoke-test the submitted APK	13
10 Verification and Quality Evidence	14

10.1 Automated test results	14
10.2 Risk-based coverage assessment	14
10.3 APK inspection	15
11 Work Division and Integration	16
11.1 Collaboration model	16
11.2 Integration work	16
12 Completion Self-Assessment	17
12.1 Course deliverable compliance	17
12.2 Assessment against grading dimensions	17
13 Limitations, Risks, and Next Work	18
13.1 Prioritized roadmap	18
14 Conclusion	19
References	19
A Requirement-to-Implementation Traceability	19
B Submission Packaging Checklist	20

1 Project Topic and Product Problem

1.1 Problem statement

Common small personal objects such as keys, wallets, earphones, chargers, access cards, and reusable cups frequently change location but the need to know where they are rarely justify a dedicated hardware tracker. In daily life, people often vaguely remember seeing an object without knowing their specific time and location.

TRACE addresses the question: *when and where did the application most recently recognize this specific enrolled object?* Real-time tracking is out of the scope of the project. To start, the user first needs to capture a reference image, selects the relevant rectangular region, and assigns a tag. On a later capture, the application searches the scene for matching visual features. A confirmed match creates a sighting containing time, confidence, an optional location fix, a bounding box, and encrypted visual evidence. A cloud-vision solution can reduce local computation, but it introduces network dependency and requires uploading images that may reveal private rooms, workspaces, or documents. Therefore, our solution is to run them locally, which also restrict the recognition power to trade for the users' privacy

1.2 Measurable project objectives

ID	Objective	Acceptance evidence	Status
O1	Install on common Android devices	APK declares minSdk 24 and launches through an exported main activity	Completed
O2	Enroll an object from one capture	Camera capture, rotation, ROI validation, quality gate, embedding, encrypted persistence	Completed
O3	Recognize enrolled objects locally	Packaged ONNX models, model metadata checks, thresholding, multi-object deduplication	Completed
O4	Retrieve the latest sighting	Persistent sighting graph, two-minute and 30 m deduplication, last-seen query	Completed
O5	Protect sensitive local data	AES-256-GCM, Android Keystore, context-bound AAD, disabled backup	Completed
O6	Demonstrate production-scale accuracy	Multi-device dataset, calibrated threshold, precision/recall and latency distribution	Partial

2 Target Users and Requirements

2.1 Primary user groups

User group	Typical context	Primary need	Design consequence
Students and office workers	Objects move between home, classrooms, laboratories, and desks	A fast record of the last observation without hardware tags	Short interaction path and acceptable performance on mid-range phones
Users with many similar items	Several objects share a broad category but differ as instances	User-defined names and account-isolated object histories	Duplicate-tag prevention and explicit reference selection
Privacy-conscious users	Captures may contain private environments	No mandatory image upload; encrypted storage at rest	On-device inference, no cloud dependency, backup disabled
Occasional users	The application is opened only when recording or searching	Immediate orientation and useful empty states	Shallow navigation and actionable error messages

2.2 Primary usage scenario

1. The user creates a local account or signs in. An email is normalized, the password is validated, and the active account session is stored locally.
2. In the **Remember** tab, the user selects tagging mode, grants camera access, captures an image, adjusts the ROI, enters a unique tag of at most 80 characters, and saves it.
3. Enrollment rejects an empty image, invalid ROI, an ROI below one percent of image area, unsuitable image quality, or a duplicate normalized tag.
4. In recognition mode, the user captures a later scene. The application evaluates all references that belong to the active account.
5. For each valid match, the application obtains a location only if permission is available, records or merges a sighting, and emits local feedback and an optional notification.
6. The **Find** tab displays the latest recorded time, confidence, and location. An object without a sighting produces an explicit empty state rather than fabricated data.

2.3 Functional and quality requirements

ID	Requirement	Priority
FR-01	Register, authenticate, and end a local account session; partition object data by account ID	Must
FR-02	Capture an image, preserve camera orientation, and allow normalized ROI adjustment	Must
FR-03	Create an encrypted object reference with a unique tag and compatible embedding metadata	Must
FR-04	Recognize one or more enrolled objects in a scene and deduplicate results by object ID	Must
FR-05	Record, deduplicate, query, and display last-seen information and timeline data	Must
QR-01	Complete the primary workflow offline and avoid a production backend dependency	Must
QR-02	Keep image decoding, inference, file operations, and database access off the UI thread	Must
QR-03	Provide confidentiality and integrity for sensitive data at rest	Must
QR-04	Support Android API 24 and target API 36 through a reproducible command-line build	Must
QR-05	Continue core processing when location or notification permission is denied	Must

3 Application Architecture

3.1 Architectural style

The delivered application is a modular Android monolith. The `:app` module owns the composition layer and device-facing user interface. Feature modules own use cases. `:core:contracts` defines stable interfaces and data-transfer models, while `:core:database` owns the Room schema and data-access interfaces. Hilt binds implementations at the application boundary. A feature module communicates with another feature through contracts rather than importing its implementation.

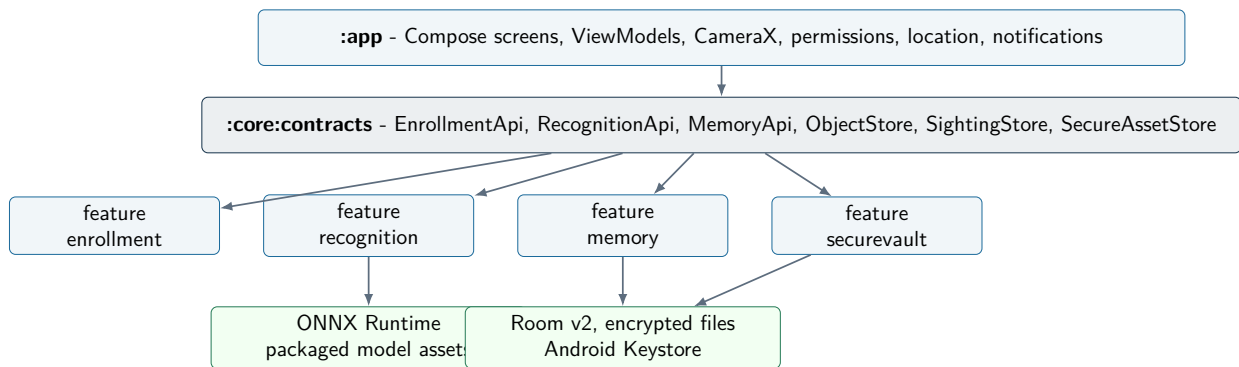


Figure 1: Runtime architecture of the submitted APK. The prototype server is outside this execution path.

3.2 Module responsibilities

Module	Owned responsibility	Excluded responsibility
<code>:app</code>	Screens, state, navigation, camera, location, permission, notification adapters	SQL, cipher primitives, inference internals
<code>:core:contracts</code>	Cross-module interfaces, immutable models, typed results and error codes	UI and persistence implementations
<code>:core:auth</code>	Input validation, password verifier, active account session	Object and sighting records
<code>:core:database</code>	Room entities, DAOs, indexes, foreign keys, schema export	Recognition and deduplication policy
<code>:feature:enrollment</code>	Tag and ROI validation, visual encoding, object-reference creation	Camera UI and SQL
<code>:feature:recognition</code>	Decode, rotate, quality analysis, SSD detection, embeddings, similarity, merge	Sighting persistence
<code>:feature:memory</code>	Sighting creation, spatial-temporal deduplication, last-seen and timeline	Room and AES implementation details
<code>:feature:securevault</code>	AES-GCM, Keystore integration, encrypted field/file stores	Product navigation and capture workflow

3.3 Control flow and failure propagation

The asynchronous contract has explicit success and failure variants. ViewModels translate typed failures into UI state instead of allowing implementation exceptions to cross the presentation

boundary. CPU-intensive processing runs on the coroutine default dispatcher; persistence uses the I/O dispatcher. Enrollment and memory services perform compensating deletion when an asset file has been written but a later database operation fails.

3.4 Integrated backend service

During development, we used an independent backend in order to encourage fast and individual progress. However, the Android final submission has no production API URL, JWT flow, image-upload path, nor Firebase client. Excluding these components reduces deployment failure and ensures that the assessed workflow remains available offline, trading the multi-device synchronization and remote recovery for the robustness and privacy.

4 Enrollment and Recognition Pipeline

4.1 Reference enrollment

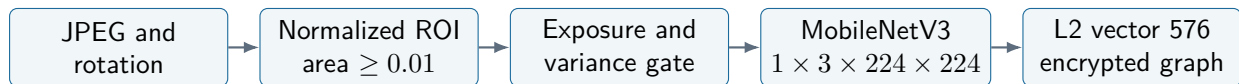


Figure 2: Data path for enrollment.

The image is decoded with a maximum working dimension of 2,048 pixels, rotated from capture metadata, and cropped using normalized coordinates. The quality estimator samples a 96-pixel representation. It rejects a mean brightness below 25, above 230, or intensity variance below 100. These checks prevent nearly empty, severely underexposed, or overexposed references from entering the comparison set.

An accepted crop is resized to 224 by 224 pixels. RGB channels are converted to NCHW float32 and normalized with ImageNet parameters:

$$x'_c = \frac{x_c/255 - \mu_c}{\sigma_c}, \quad \mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225).$$

MobileNetV3-Small produces a 576-dimensional feature vector, which is normalized to unit L2 norm. The stored reference includes model name, model version, vector dimension, ROI, image asset ID, and quality score. These fields make incompatible embedding spaces detectable rather than silently comparable.

Before getting stored, the tag is trimmed, Unicode-normalized to NFC, internal whitespace is collapsed, and the result is compared case-insensitively against existing references. A duplicated tag is rejected with an actionable message. If object graph insertion fails after the encrypted image has been written, the image asset is deleted as compensation.

4.2 Dual-path scene recognition

Recognition evaluates two logical candidate paths:

1. **Full-image path.** A MobileNet embedding is extracted from the complete scene. This preserves a fallback when SSD returns no box or its inference fails.
2. **Detection-crop path.** SSD MobileNet v2 accepts a uint8 NHWC tensor with shape $1 \times 300 \times 300 \times 3$. The implementation keeps at most 20 boxes with confidence at least 0.30, adds 10 percent context padding, crops each region, and extracts a MobileNet embedding.

For candidate vector q and reference vector r , the similarity is

$$s(q, r) = \frac{q \cdot r}{\|q\|_2 \|r\|_2}.$$

Both vectors are L2-normalized, so the implementation reduces this calculation to a dot product. References are filtered by model name, version, and dimension before comparison. A score below `minimumSimilarity` produces `UNKNOWN`. Matches are sorted by score, deduplicated by object ID, and limited to 1 through 20 results. If no match survives, the API returns an explicit unknown result rather than an ambiguous empty response.

4.3 Algorithmic strengths and limits

The detection-crop path reduces background interference for small objects in cluttered scenes, while the full-image path makes recognition less brittle when the detector does not propose a useful region. However, MobileNetV3-Small is a general feature extractor, thus objects with limited texture or multiple visually similar instances may remain difficult. A production accuracy claim requires a representative dataset and calibrated operating threshold.

5 Memory Logic and Persistent Data

5.1 Room schema

The local schema is version 2 and contains five tables. Sensitive values are sealed before they reach Room. Large images are stored as encrypted files in private application storage; Room retains only relative paths and envelope metadata.

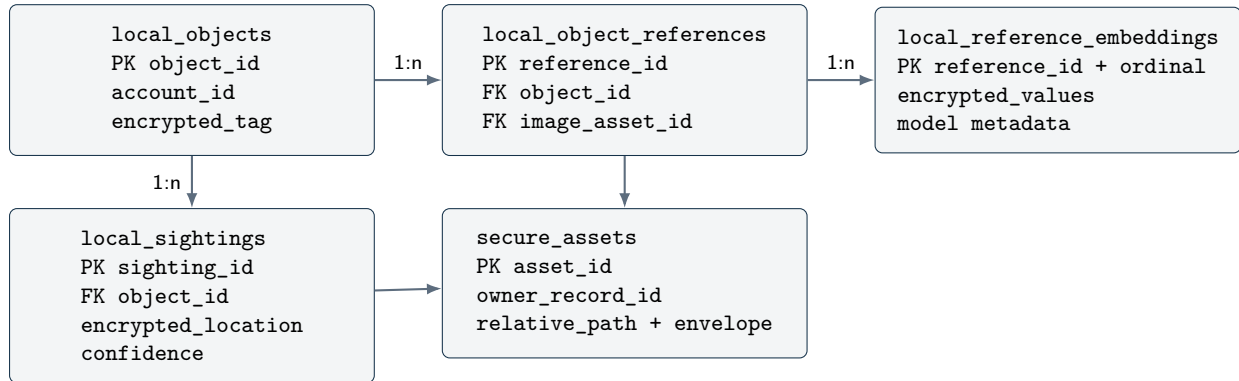


Figure 3: Local persistence graph and encrypted asset linkage.

Table	Primary role	Sensitive field treatment
local_objects	Account-scoped identity, revision, timestamps	Tag stored as encrypted BLOB
local_object_references	ROI, quality, image link, object relationship	Image content referenced through secure asset
local_reference_embeddings	Ordered model vectors and compatibility metadata	Float vector encoded and encrypted
local_sightings	Observation time, confidence, evidence, sync marker	Exact location encrypted when present
secure_assets	File ownership, relative path, MIME type, key and envelope metadata	File at path contains ciphertext only

Indexes cover account ID, object ID, object-time queries, asset ownership, and sync status. Foreign keys cascade object graph deletion while asset-file lifecycle is coordinated by the secure store adapter. Insertions of an object, reference, and embeddings use a Room transaction.

5.2 Sighting creation and deduplication

Only recognition results with **MATCHED** status and a non-null object ID reach the memory service. Confidence must fall in $[0, 1]$, and the referenced object must exist. The new event is compared with the latest sighting for the same object. Two events are merged when

$$0 \leq \Delta t \leq 120 \text{ s} \quad \wedge \quad (\text{a location is unavailable} \vee d_{\text{Haversine}} < 30 \text{ m}).$$

On merge, the new time the new location replaces the old one when available, and the greater confidence is retained, while the original ID is kept. Otherwise, a new UUID and encrypted evidence asset are created. Timeline queries accept limits from 1 to 100 and return the newest observation first.

5.3 Consistency boundary

Room can atomically update relational records but cannot include file-system writes in the same ACID transaction. The current service uses compensating deletion when a later insert fails. This is adequate for the project scope but does not cover process termination between the two operations. A production revision should add a pending-write journal and an orphan-asset reconciliation job, verified with fault injection.

6 Security and Privacy Design

6.1 Threat model and trust boundary

The Secure Vault is designed for a storage-compromise adversary. Protected data comprises reference and sighting images, object tags, visual embeddings, exact locations, password verifiers, and cryptographic keys. The adversary may copy the complete Room database and asset directory, modify or truncate stored bytes, or replace one valid row or asset with another. Consequently, bytes read from disk are attacker-controlled until cryptographic verification succeeds.

The scoped adversary cannot extract the AES secret key from Android Keystore, read plaintext already resident in TRACE process memory, or execute code inside the trusted application process. Root-level runtime instrumentation, screen capture, and a fully compromised operating system are therefore residual risks rather than properties solved by at-rest encryption. Android Keystore hardware backing is device-dependent and is not claimed as universal.

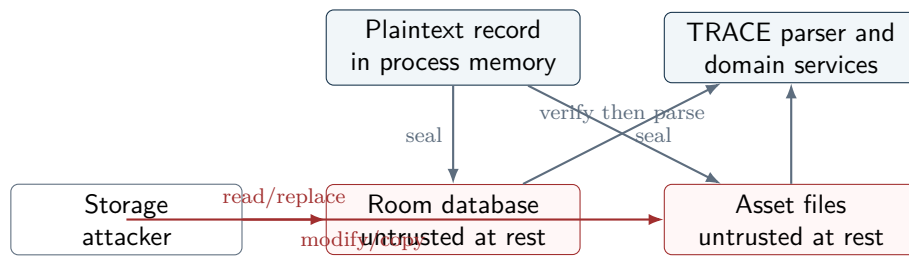


Figure 4: Secure Vault trust boundary. Persistent bytes are untrusted inputs; plaintext is admitted only after authenticated decryption.

Without the vault, the storage relation would be

$$B_{\text{disk}} = \text{Serialize}(P), \quad P' = \text{Deserialize}(B_{\text{disk}}),$$

which gives a disk attacker both confidentiality loss and direct control over the deserializer input. TRACE instead inserts authenticated decryption between storage and parsing.

6.2 Confidentiality under complete storage disclosure

`AndroidKeystoreKeyProvider` generates a 256-bit AES key under alias `trace_secure_vault_v1`. Application code receives a non-exportable key handle, while the key bytes are not serialized into Room or asset files. The key specification permits only AES-GCM encryption/decryption, requires randomized encryption, and uses no padding.

For plaintext P , fresh 96-bit nonce N , key K , and associated data A , the vault computes

$$(C, T) = \text{GCM.Enc}_K(N, P, A).$$

The Android cryptographic provider generates N during cipher initialization. This detail is necessary on API 24, where a caller-supplied IV is rejected for Keystore keys configured to require randomized encryption. The implementation rejects any returned nonce whose length is not 12 bytes.

The logical envelope is

$$E = (v_e, \text{algorithm}, \text{keyId}, N, C \| T),$$

where $v_e = 1$, the algorithm identifier is AES-256-GCM, and T is 128 bits. Encrypted scalar values in Room contain the encoded complete envelope. For large image assets, $C||T$ is stored in a private `.bin` file while version, algorithm, key ID, nonce, ownership, and MIME metadata are stored in the `secure_assets` row. Copying both stores therefore yields the complete envelope but not K , so the scoped offline attacker cannot recover P .

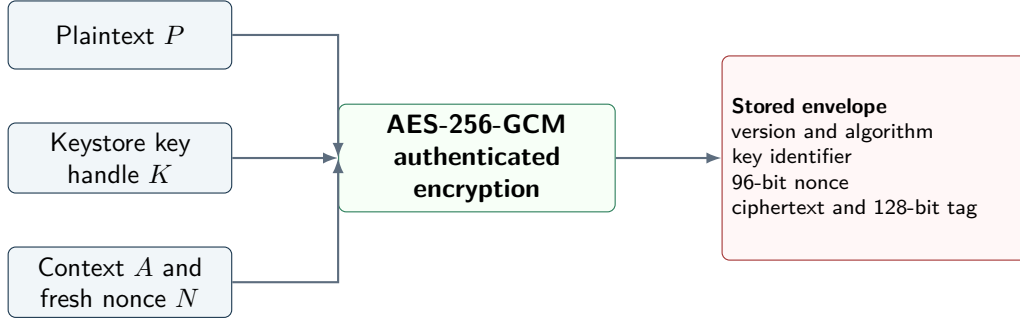


Figure 5: Confidentiality path. The envelope is complete for decryption but intentionally excludes secret-key material.

6.3 Tamper detection and fail-closed release

AES-GCM authenticates both ciphertext and associated data. At a conceptual level, GCM derives the hash subkey and tag as

$$H = E_K(0^{128}), \quad T = \text{MSB}_{128}(E_K(J_0) \oplus \text{GHASH}_H(A, C)),$$

where J_0 is derived from N . TRACE delegates these primitives to the platform JCA provider; it does not implement GHASH manually. During `open`, the provider recomputes the tag over the received nonce, expected AAD, and ciphertext. A changed bit, injected byte, truncated payload, wrong key, or changed context causes verification failure:

$$\text{GCM.Dec}_K(N, C, T, A) = \begin{cases} P, & \text{if authentication succeeds,} \\ \perp, & \text{otherwise.} \end{cases}$$

`AEADBadTagException` becomes `VaultFailureReason.AUTHENTICATION_FAILED`; no unauthenticated plaintext or compatibility fallback is returned. Before invoking GCM, the envelope decoder also bounds the total payload to 16 MiB, rejects trailing bytes, restricts metadata fields to 256 bytes, validates version and algorithm, requires a 12-byte nonce, and requires at least a 16-byte ciphertext/tag payload.

6.4 Context binding against record substitution

Ciphertext integrity alone is insufficient. An attacker could move an unchanged, valid envelope from record A to record B; its tag would still verify if both records used identical external context. TRACE prevents this semantic substitution by authenticating an application-constructed `VaultContext`. Its byte encoding is deterministic and length-prefixed:

$$A = \text{Encode}(0x54524358, v_s, \text{accountId}, \text{recordId}, \text{field}),$$

where the magic value separates this encoding domain and v_s is the schema version. The reader reconstructs A from the record it expects to load rather than trusting context carried by the stored envelope.

Let E_A be created with A_A for record A. If the same envelope is placed in record B, then $A_A \neq A_B$ whenever the account, record identity, field identity, or schema version differs. Therefore

$$\text{GCM.Dec}_K(N_A, C_A, T_A, A_B) = \perp.$$

This binds plaintext to its intended semantic slot, not merely to its ciphertext bytes. Embedding contexts additionally include ordinal, model name, and model version. Asset contexts include owner record ID, asset UUID, asset type, and MIME type; a valid reference image envelope therefore cannot be reinterpreted as sighting evidence.

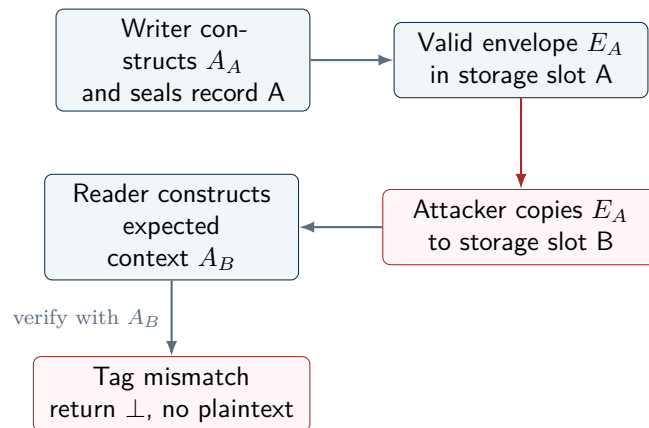


Figure 6: Record-swapping defense. A cryptographically valid envelope is rejected when presented under a different application context.

6.5 Storage adapters and operational safeguards

`RoomObjectStore` seals tags and each 576-dimensional embedding before insertion. `RoomSightingStore` separately seals exact locations. The encrypted asset adapter writes image ciphertext to a private, backup-excluded directory. Assets use the `secure-assets-v1` subdirectory; a UUID-only filename policy and canonical-parent check prevent path traversal. File publication uses a temporary file, `fd.sync()`, and rename, and removes the file if the following Room insertion fails. Reads first enforce account, owner-record, and asset-type equality, then reconstruct the expected context and authenticate the bytes.

The manifest disables Android backup through `allowBackup=false` and `fullBackupContent=false`, reducing accidental ciphertext restoration without the corresponding Keystore key. These controls do not make Room and the file system one ACID transaction: process death between operations can still leave an orphan file or row. A write journal and reconciliation worker remain recommended.

6.6 Local credential handling

The application does not store plaintext passwords. Registration creates a random 16-byte salt and derives a 256-bit verifier using PBKDF2-HMAC-SHA1 with 210,000 iterations. Verifier comparison is constant-time. Account preference keys use SHA-256 of the normalized email, while the session retains only the active account ID.

Security qualification. This mechanism is suitable for local course-project authentication, but it is not equivalent to hardware-backed user presence. PBKDF2-HMAC-SHA1 is retained for compatibility rather than as a preferred new design. A subsequent version should evaluate PBKDF2-HMAC-SHA256 or Argon2id, add retry backoff, and optionally require the device credential or biometric authentication before releasing decryption keys.

6.7 Threat-control matrix

Threat	Current control	Residual risk
Copied application files or backup	Android sandbox, backup disabled, AES-GCM ciphertext	A rooted device can observe process memory
Ciphertext moved between records	AAD binds account, record, field, and schema version	Rotation and whole-graph migration still need tests
Ciphertext or meta-data tampering	GCM authentication and fail-closed decryption	No privacy-preserving corruption telemetry
Image disclosure over network	No upload on final runtime path; workflow is offline	Merged manifest and dependencies require release audit
Offline password guessing	Per-account random salt and 210,000 PBKDF2 rounds	No attempt throttling or device-credential gate
Restored ciphertext without its key	Backup disabled to avoid mismatched data-key restoration	No secure user-controlled export and recovery path

7 User Interface and Device Integration

7.1 Screen structure and state flow

Before authentication, **TraceApp** displays either an initialization indicator or the account form. After authentication, **TraceShell** exposes four bottom-navigation destinations: **Home**, **Remember**, **Find**, and **Profile**. A small navigation history supports predictable back behavior. A successful enrollment or recognition increments **dataRevision**, causing object state to refresh without giving screens direct database access.

Screen	Primary task	Handled states
Authentication	Register or sign in to a local account	Initializing, form validation, duplicate account, invalid credential, success
Home	Summarize stored objects and provide task shortcuts	Loading, empty collection, populated collection
Remember	Switch tagging/recognition mode, capture, adjust ROI, submit	Permission, no image, busy, failure, saved, unknown, single or multiple matches
Find	Select an object and read latest sighting	No objects, object without sighting, complete time/location/confidence result
Profile	Display current account and terminate session	Authenticated account and logout action

7.2 Runtime permissions

Camera permission is essential because image capture is the product input, and the manifest declares a camera feature as required. Fine and coarse location are optional. Location is requested only for a recognition event and the sighting remains valid with a null location when permission is denied. Notification permission is optional on Android 13 and later; denial suppresses the notification without blocking inference or persistence.

7.3 ROI interaction and duplicate-submission control

ROI coordinates are normalized to $[0, 1]$, making them independent of preview and image resolution. Transform logic has unit tests for scale, crop, and camera orientation, while the overlay gesture has an instrumented test. The tag input is capped at 80 characters in UI state and validated again at the service boundary. After a successful enrollment, `enrollmentSaved` prevents accidental double submission of the same capture.

7.4 Accessibility status

Compose and Material 3 provide baseline semantic roles and touch targets, but the repository does not contain a complete TalkBack audit, 200 percent font-scale test, switch-access test, or multi-size screenshot suite. Consequently, accessibility is reported as an open verification item rather than assumed complete from framework choice alone.

8 Technology Stack and Engineering Choices

Component		Repository	version	Technical role
Kotlin		2.3.21		Null-safe implementation language for UI, domain logic, coroutines, and Gradle DSL.
Android Gradle Plugin		8.13.2		Build graph, lint, packaging, R8, resource shrinking, and ABI configuration.
Jetpack BOM	Compose	2026.06.01		Declarative state-driven UI and semantics-based UI testing.
Material 3		BOM managed		Consistent controls, navigation, forms, loading, and feedback.
Lifecycle		2.10.0		Lifecycle-aware state collection through <code>collectAsStateWithLifecycle</code> .
CameraX		1.6.2		Camera2 compatibility, lifecycle-bound preview, and still capture.
Room		2.8.4		Typed schema and DAO layer, indexes, foreign keys, migration, and schema export.
Hilt		2.58		Application composition root and interface-to-implementation binding.
Kotlin Coroutines		1.10.2		Structured concurrency and explicit CPU/I/O dispatch boundaries.
ONNX Runtime	Android	1.26.0		Offline execution of packaged neural-network models.
MobileNetV3-Small		Model asset, version 1		576-dimensional visual feature extraction at 224 by 224 input.
SSD MobileNet v2		Model asset, opset 13		Object proposals at 300 by 300 input for background-reduced crops.
Google Play Location		21.4.0		Optional fused location fix for confirmed sightings.
JUnit and Truth		4.13.2 / 1.4.4		JVM unit-test execution and readable assertions.
AndroidX Test		1.2.1; runner 1.6.2		Instrumented UI, Keystore, and repository tests on API 24.
JDK / bytecode		Toolchain JVM 17	21 /	Current build toolchain with conservative Android bytecode target.

8.1 On-device versus client-server processing

On-device processing removes network latency from the critical path and keeps private scene images local. It also avoids running a database or API during demonstration. The cost is a larger APK and dependence on device CPU, memory, and ABI compatibility. The current submission chooses functional independence over cross-device synchronization.

8.2 Packaging and release strategy

The release build enables R8 and resource shrinking. The course submission command produces an ARM64 APK signed with a debug certificate, which satisfies the specification's acceptance of a release or signed-debug artifact but is not suitable for public store distribution. The `bundle` command creates an AAB and uses a real release keystore only when four environment variables are supplied. No keystore or password is committed.

9 Setup, Build, and Installation

9.1 Required environment

Dependency	Requirement
Host operating system	Windows, macOS, or Linux with the required JDK and Android SDK
Java toolchain	JDK 21 LTS; Android bytecode targets Java 17
Android SDK	Platform 36 and Build Tools 35 or later
Target device	Android 7.0/API 24 or later; current submission APK targets ARM64
Network	Required only for first-time dependency resolution; not required by the installed application
Android Studio	Optional; command-line build and install are supported

9.2 SDK discovery

The root build script locates JDK 21 even if another Java version is the host default. It discovers Android SDK through `ANDROID_HOME`, `ANDROID_SDK_ROOT`, or the uncommitted `apps/android/local.properties` file:

```
sdk.dir=C:\\Users\\YOUR_NAME\\AppData\\Local\\Android\\Sdk
```

9.3 Canonical commands from repository root

```
.\build-android.{bat,sh} build      # Build an installable debug APK
.\build-android.{bat,sh} check      # Unit tests, Android Lint, debug APK
.\build-android.{bat,sh} install    # Install on an adb-connected device
.\build-android.{bat,sh} submission # ARM64 signed-debug submission APK
.\build-android.{bat,sh} bundle     # Release AAB for store distribution
```

9.4 Install and smoke-test the submitted APK

On Windows machines:

```
adb devices
adb install -r .\apk\app-release.apk
Get-FileHash .\apk\app-release.apk -Algorithm SHA256
```

On Linux devices

```
adb devices
adb install -r ./apk/app-release.apk
sha256sum ./apk/app-release.apk
```

If the target device is not ARM64, build the debug artifact or generate the matching ABI split. After installation:

1. Create a local account, sign out, and sign in again.
2. Grant camera permission. Deny location and notification once to verify graceful degradation.
3. Enroll a textured object under good lighting with a close ROI and unique tag.
4. Capture the object in a different scene and confirm recognition feedback.
5. Open Find and verify time, confidence, and location when permission has been granted.
6. Disable Wi-Fi and mobile data and repeat the workflow to confirm offline operation.

10 Verification and Quality Evidence

10.1 Automated test results

Test group	Tests	Failures	Errors	Level
Normalized rectangle contract	2	0	0	JVM unit
Authentication validation	3	0	0	JVM unit
Camera orientation and ROI transform	6	0	0	JVM unit
Enrollment service	1	0	0	JVM unit
Cosine similarity and recognition post-processing	4	0	0	JVM unit
AES-GCM vault	7	0	0	JVM unit
UI, local auth, ROI gesture, Android Keystore	10	0	0	Instrumented, API 24
Total	33	0	0	33/33 passed

On 5 September 2026, `build-android.batchcheck` reported `BUILDSUCCESSFUL` in 38 seconds. Its 439 actionable Gradle tasks comprised 11 executed tasks, 3 cache hits, and 425 up-to-date tasks. . Connected instrumented tests are not part of `check`; the 10/10 instrumented result is taken from stored XML generated by the Android 7.0 AVD `trace_bugfix_api24`.

10.2 Risk-based coverage assessment

Failure risk	Current evidence	Missing evidence
Incorrect orientation or ROI mapping	Orientation and transform unit tests; gesture instrumented test	Physical camera OEM matrix
Incorrect match or merge result	Cosine, metadata filtering, sorting and deduplication tests	Representative precision/recall benchmark
Ciphertext loses integrity	Round trip, unique nonce, modified AAD/ciphertext, and wrong-key tests	Rotation migration and file/DB fault injection
Authentication rejects or leaks state	Validation unit tests; account isolation instrumented tests	Retry backoff and corrupted-preference cases
Unusable empty/error UI	Compose tests for authentication, home, and find	TalkBack, font-scale, and screenshot matrix
API 24 incompatibility	Full stored instrumented suite on Android 7.0	Additional API 29, 33, and 36 coverage

10.3 APK inspection

Package: com.traceapp.android **Version:** 0.1.0 (1) **minSdk/targetSdk:** 24/36
Size: 96,041,639 bytes, approximately 91.6 MiB
SHA-256: 0DCC1C3266860A0F67F4B834CA61C56BC8D57C00E60FF1A8FD47D049A488C52B

The principal size contributors are the two ONNX models and native ONNX Runtime libraries. A production release should use an AAB so the store can deliver only the required ABI.

11 Work Division and Integration

11.1 Collaboration model

The team separated four technical problems behind shared contracts, first validating algorithms in independent playground modules and then integrating them into the final Android application. Branch names, module ownership, commit history, tests, and contribution notes provide the evidence used below. Commit count is based on each member's preference of committing and thus not to be treated as a direct measure of effort.

Member	Branch / module	Primary contribution	Repository evidence
Member 1 Khuong_ Duong	<code>khuong_enrollment;</code> <code>member1-enrollment</code>	Enrollment validation, ROI pre-processing, MobileNetV3 embedding, quality scoring, algorithm tests, Android migration guidance	Commit 1936096; <code>EnrollmentAlgorithm</code> ; enrollment tests and contribution notes
Member 2 phongcp	<code>phong_recognition;</code> <code>member2-recognition</code>	Dual-path recognition, SSD boxes, contextual crop, MobileNet embedding, cosine threshold, result merge, real-scene integration test	Commit 5698241; 18 unit and one playground integration test; Android post-processing tests
Member 3 Nguyen Hoang Thuan Phat / Bix2llets	<code>Phat;</code> <code>member3-memory</code>	Last-seen timeline, tag matching, two-minute/30 m deduplication, behavioral pattern work, schema migration correction	Commits 818f14a and 302678b; memory module, documentation, merge commit 9f3fd32
Member 4 ThanDong VanHoc	<code>final; feature:</code> <code>securevault</code>	Security architecture and Secure Vault: storage threat model, AES-256-GCM envelope, Android Keystore key provider, context-bound AAD, encrypted Room fields and asset files, fail-closed error mapping, security tests; secondary final-app integration	Commits c0126a9, 45d0aac, and 7005330; <code>CryptoVault</code> , <code>EncryptedSecureAssetStore</code> , encrypted Room stores, seven JVM vault tests, Android Keystore instrumented test

11.2 Integration work

Each feature depends on `core:contracts`. Integration replaced desktop ONNX Runtime with the Android runtime, adapted bitmap processing, replaced playground SQLite access with encrypted Room stores, and connected ViewModel state to Compose. Review focused on embedding metadata compatibility, account scoping, asset rollback, permission fallback, and multi-object result handling.

12 Completion Self-Assessment

12.1 Course deliverable compliance

Required item	Assessment	Evidence and qualification
Installable Android application, API 24+	Completed	Valid package and SDK declarations, successful build, and stored API 24 instrumented results.
At least three or four connected screens	Completed	Authentication plus Home, Remember, Find, and Profile with shared state and navigation.
Persistent local data	Completed	Room v2, exported schema, foreign keys, indexes, encrypted fields and files, account partitioning.
External source or device capability	Completed	CameraX is mandatory; fused location and local notifications are optional integrations.
Architecture, lifecycle, asynchronous work	Completed	Contract modules, Hilt, ViewModel/StateFlow, lifecycle-aware collection, Default/IO dispatchers.
Release or signed-debug APK	Completed	apk/app-release.apk is an ARM64 signed-debug artifact permitted by the specification.
Technical report of 10 to 30 pages	Completed	This one-column LaTeX report covers every required topic and is render-verified.
Demonstration video of 5 to 10 minutes	Completed	No video link is present in the repository snapshot; completion is an external submission task.
README with identities and demo data	Completed	Build documentation exists; verified student IDs, official names, and video link are unavailable.

12.2 Assessment against grading dimensions

Dimension	Level	Technical basis
Functional completeness, 30 percent	High	Complete offline auth-enroll-recognize-find workflow, installable APK, and automated evidence; multi-device soak testing remains absent.
Technical quality, 25 percent	High	Explicit boundaries, typed results, Room graph, dispatcher separation, context-bound encryption; known debt in KDF choice and file/DB atomicity.
User interface and experience, 20 percent	Good	Shallow navigation, permission fallback, busy/empty/error states, and duplicate-submit control; accessibility matrix incomplete.
Originality and complexity, 15 percent	High	ROI enrollment, dual-model on-device matching, spatial-temporal memory, and encrypted local persistence form a non-template system.
Report and collaboration, 10 percent	High	Technical ownership and traceability are documented; having video and official personal metadata.

13 Limitations, Risks, and Next Work

Current limitation	Engineering impact	Recommended next action
No multi-device recognition benchmark	Precision, recall, false-match rate, and p95 latency cannot be claimed reliably	Collect 30 to 50 objects under varied lighting and pose; retain a holdout set and report PR/ROC curves
One primary reference per object	Recognition is sensitive to occlusion and viewpoint change	Support three to five controlled references and define score aggregation
Large APK	Slower distribution and risk of channel size limits	Publish AAB, retain ABI splits, evaluate quantization against an accuracy budget
Room and file system are not one transaction	Process death can leave an orphan file or pending row	Add sufficient logging for recovery
Key rotation is not operationalized	Envelope key IDs exist but records are not automatically re-encrypted	Implement checkpointed background migration with roll-back and power-loss tests
No authentication retry backoff	A local attacker can attempt many passwords	Add exponential back-off and optional device-credential/biometric gating
No backup or synchronization	Device loss implies data loss	Design user-controlled encrypted export or end-to-end encrypted synchronization
Accessibility is not audited	High font scale or assistive technology may expose layout defects	Test TalkBack, 200 percent font scale, contrast, switch access, and size matrix

13.1 Prioritized roadmap

1. **P0 - reliability:** run emulator CI on API 24 and 36, measure p50/p95 inference latency, test process death between file and Room operations, and complete the accessibility audit.
2. **P1 - recognition quality:** support multiple references, calibrate the threshold against a holdout set, and quantify any accuracy change before model quantization.
3. **P2 - product continuity:** add secure export or end-to-end encrypted synchronization, checkpointed key rotation, and self-service data deletion.

14 Conclusion

TRACE delivers an Android system for recording the last observed location and time of personal objects. Its main technical contribution is an end-to-end local data path: CameraX captures an image; ROI and quality rules constrain input; two ONNX models generate proposals and visual embeddings; cosine matching produces object detections; the memory service applies temporal and spatial rules; Room stores the relational graph; AES-GCM and Android Keystore protect sensitive values; and Jetpack Compose presents the result through lifecycle-aware state. Therefore, the final application satisfies the central functional and technical requirements and is supported by an inspected APK, a successful build, and unit and instrumented test evidence.

References

1. TRACE repository, final branch: [README.md](#), [Requirement.md](#), [apps/android](#), [docs](#), Git history, build output, and test reports; inspected 5 September 2026.
2. Android Developers, [Jetpack Compose](#), [Room](#), and [Android Keystore documentation](#).
3. Microsoft, [ONNX Runtime Java](#) and [Android documentation](#).
4. National Institute of Standards and Technology, [SP 800-38D: Galois/Counter Mode](#).

A Requirement-to-Implementation Traceability

Requirement	Primary implementation	Verification evidence
FR-01 Au- thentication	<code>core/auth/AuthRepository.kt</code> , <code>AuthValidation.kt</code> , <code>ui/auth</code>	Validation unit tests; <code>LocalAuthRepository</code> and <code>AuthScreen</code> instrumented tests
FR-02 Cam- era and ROI	<code>CameraPreview.kt</code> , <code>RoiOverlay.kt</code> , <code>RoiTransform.kt</code> , <code>ScanScreen.kt</code>	<code>CameraOrientation</code> , <code>RoiTransform</code> , and <code>RoiOverlayGesture</code> tests
FR-03 Enroll- ment	<code>feature/enrollment/</code> <code>EnrollmentService.kt</code> ; VisualEncoder contract	<code>EnrollmentServiceTest</code> and manual en- rollment smoke path
FR-04 Recog- nition	<code>OnDeviceVisualEngine.kt</code> , <code>CosineSimilarity.kt</code> , packaged models	<code>CosineSimilarity</code> and <code>RecognitionPost-</code> <code>Processing</code> tests; playground scene inte- gration
FR-05 Mem- ory	<code>feature/memory/MemoryService.</code> <code>kt</code> ; Room sighting store and DAO	<code>FindScreen</code> test, rule inspection, and playground memory tests
QR-01 Offline runtime	Android <code>README</code> , final manifest, application dependency graph	Offline smoke procedure and absence of production API configuration
QR-02 Asyn- chronous work	<code>ViewModel</code> coroutines; <code>Dispatchers.</code> <code>Default</code> and <code>Dispatchers.IO</code>	Build/lint and code review of dispatch boundaries
QR-03 Secu- rity	<code>CryptoVault.kt</code> , <code>AndroidKeystoreKeyProvider.kt</code> , encrypted stores	<code>AesGcmVault</code> unit tests and Android- <code>KeystoreVault</code> instrumented test
QR-04 Com- patibility	<code>app/build.gradle.kts</code> , root build scripts, APK manifest	aapt package inspection and API 24 in- strumented result
QR-05 Per- mission fallback	Manifest, <code>LocationReader.kt</code> , noti- fier and settings adapters	UI branch inspection and manual denial smoke path

B Submission Packaging Checklist

No.	Item	Current state
1	Name the directory and ZIP with ascending student IDs	Student IDs required
2	Add IDs, official names, title, build steps, test account, and video link to README	Personal metadata incomplete
3	Include source while excluding build, <code>.gradle</code> , <code>node_modules</code> , and <code>.idea</code> ; retain Git history	Verify during packaging
4	Include <code>apk/app-release.apk</code> and install it on Android API 24 or later	Artifact present; final physical test required
5	Place this rendered report at <code>report/report.pdf</code>	Copy during package assembly
6	Add <code>video/demo-link.txt</code> for the 5 to 10 minute demonstration	Missing from repository snapshot
7	Confirm Drive or YouTube access in an incognito browser	Not yet verified
8	Have one representative submit the form and retain the confirmation email	Complete at submission time